COLLEGENOTICE – DIGITAL NOTICE BOARD SYSTEM

A MINI PROJECT REPORT

Submitted by

NITHIN MATHEW VARGHESE

MLM25MCA-2041

To

the APJ Abdul Kalam Technological University

in Partial fulfillment of the requirements for

the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. MINNU V B

(Assistant Professor, Department of Computer Applications)



DEPARTMENT OF COMPUTER APPLICATIONS

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO – PSO – SDG

LIST OF PROGRAMME OUTCOMES (POs)

PO1 (Foundation Knowledge): Apply programming and data management foundations to develop a college notice publishing and retrieval platform.

PO2 (Problem Analysis): Examine missed institutional announcements, inaccessible physical notices, unstructured messages and inconsistent notice expiry.

PO3 (Development of Solutions): Design an integrated solution for administrator publication, student notice discovery, filtering and notifications.

PO4 (Modern Tool Usage): Use Django, template technologies, source-based language assistance, database management and documented software practices.

PO5 (Individual and Teamwork): Organize development and communicate design and results during project reviews.

PO6 (Project Management): Organize development into manageable modules and prioritize publication, security and accessibility.

PO7 (Ethics): Control private administration functions and responsible access to institutional information.

PO8 (Life-long Learning): Extend the platform for new notification mechanisms, languages and evolving campus requirements.

1. **MAPPING WITH PROGRAMME OUTCOMES (POs)**

Sl. No	POs Addressed	Relevance to Project
1	PO1 – Foundation Knowledge	Uses programming, Django models and relational structures for notice storage and retrieval.
2	PO2 – Problem Analysis	Addresses missed announcements, manual notice expiration and inadequate departmental filtering.
3	PO3 – Development of Solutions	Provides publication, discovery, category search, bookmarks and notice lifecycle management.
4	PO4 – Modern Tool Usage	Uses Django, JavaScript, responsive interfaces and translation/assistant helpers.
5	PO5 – Individual and Teamwork	Supports modular planning, student-administrator role analysis and communicated design.
6	PO6 – Project Management	Separates notice creation, student discovery and management into testable workflows.
7	PO7 – Ethics	Restricts notice changes to authorized roles and distinguishes public from private operations.
8	PO8 – Life-long Learning	Enables future extension to notification channels, accessibility and multilingual support.

2. MAPPING WITH PROGRAMME SPECIFIC OUTCOMES (PSOs)

Sl. No	PSOs Addressed	Relevance to Project
1	PSO1 – Advanced Technologies	Applies Django, modular interfaces and assistive text features to campus information workflows.
2	PSO2 – Sustainable Computing	Digital notices reduce repeated paper postings and provide an extensible campus service.

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs)

SDG No	SDG Addressed	Relevance to Project
SDG 4	Quality Education	Improves access to institutional academic notices and deadlines.
SDG 9	Industry, Innovation & Infrastructure	Provides a structured digital channel for campus-wide notice management.
SDG 12	Responsible Consumption & Production	Can reduce paper-based notice duplication and physical replacements.
SDG 16	Peace, Justice & Strong Institutions	Supports organized, accountable institutional communication.
SDG 10	Reduced Inequalities	The English–Malayalam interface option reduces a language barrier for citizens and collection staff by making core service navigation and instructions available in Malayalam.

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and lifelong learning.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable and sustainable energy.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth.

SDG 9 – Industry, Innovation and Infrastructure: Build resilient infrastructure and foster innovation.

SDG 10 – Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make human settlements inclusive, safe, resilient and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use oceans, seas and marine resources.

SDG 15 – Life on Land: Protect and restore terrestrial ecosystems and biodiversity.

SDG 16 – Peace, Justice and Strong Institutions: Build effective, accountable and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen implementation and global partnerships

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026

DECLARATION

I hereby declare that the mini project entitled "COLLEGENOTICE – DIGITAL NOTICE BOARD SYSTEM" submitted to the Department of Computer Applications, Mangalam College of Engineering, Ettumanoor, is an authentic record of my work carried out under the guidance of Ms. Minnu V B, Assistant Professor, Department of Computer Applications. The report has not been submitted elsewhere for any other degree.

Name & Signature of Student

Date: NITHIN MATHEW VARGHESE

Place:

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



CERTIFICATE

Certified that the report entitled **"COLLEGENOTICE – DIGITAL NOTICE BOARD SYSTEM"** *submitted by* ***NITHIN MATHEW VARGHESE, MLM25MCA-2041*** *to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the* ***DEGREE OF MASTER OF COMPUTER APPLICATIONS*** *by APJ* ***ABDUL KALAM TECHNOLOGICAL UNIVERSITY*** *is a bonafide record of the MINI PROJECT work carried out by him under our guidance and supervision during the academic year 2025-27*

PROJECT GUIDE

Ms. Minnu V B

Assistant Professor

Department of Computer Applications

PROJECT COORDINATOR

Ms. Athira R Kurup

Assistant Professor

Department of Computer Applications

HEAD OF THE DEPARTMENT

Ms. Divya S B

Assistant Professor

Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic "CollegeNotice-Digital Notice Board System".

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering, for providing the facilities and support to complete my Project successfully.

I thank and express my sincere gratitude to **Ms. Divya S B**, HOD& Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support, which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide**, Ms. Minnu V B**, Assistant Professor, Department of Computer Applications, for her valuable suggestions, guidance, and encouragement, which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties, especially Project Coordinator, **Ms. Athira R Kurup**, Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, who gave me the necessary guidance, permission, and resources required to complete the project presentation and report.

Finally, I would like to express my heartfelt thanks to my parents for their continuous support, encouragement, and motivation throughout the completion of this Project.

NITHIN MATHEW VARGHESE

MLM25MCA-2041

ABSTRACT

CollegeNotice-Digital Notice Board System is a web-based institutional digital notice board designed to organize college announcements in one accessible application. The system supports authenticated administration, department-based publication, notice categorization, expiry handling, searching, student bookmarks, notifications and audit-friendly record management.

The software is implemented using Python and Django, with HTML, CSS and JavaScript templates and a relational data model. Students can view notices and filter announcements by category, department and keywords, while authorized staff can manage notice lifecycles and attached documents.

The application includes a source-grounded notice assistant and a local English–Malayalam translation facility. These features support students who need concise explanations of published notices and language assistance, while remaining dependent on available notice content and language resources.

Student and administrator workflows are separated with access control, and notice information can be used through a responsive interface. The system reduces dependence on physical notice boards, allows students to check academic information away from campus and helps administrators manage current and expired notices.

The source-code-backed modules and screenshots demonstrate the implemented notice management, student reading, smart filtering, translation and administrative interface. Proposed cloud integrations from early presentation material are distinguished from features confirmed by the supplied application.

The report documents the problem context, requirements, related work, architecture and data-flow models, implemented modules, tests, observed results, limitations and future enhancements.

Keywords: Digital Notice Board, CollegeNotice, Django, Notice Management, Department Filtering, Student Notification, Malayalam Translation, Role-Based Access Control, PWA

.

Mapping with Sustainable Development Goals	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 12 – Responsible Consumption and Production SDG 16 – Peace, Justice and Strong Institutions

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
Chapter 1	**INTRODUCTION**	1
1.1	Background	1
1.2	Motivation	1
1.3	Scope And Objectives	1
1.4	Problem Statement	2
1.5	Institutional Communication Context	2
1.6	Stakeholder Responsibilities	3
1.7	Objective Traceability	3
Chapter 2	**REQUIREMENTS AND ANALYSIS**	5
2.1	Existing System And Limitations	5
2.2	Solution Overview	5
2.3	Functional Requirements	5
2.4	Non-Functional Requirements	6
2.5	Feasibility Study	6
2.6	Risks And Constraints	7
2.7	User And Role Requirements	8
2.8	Notice Validation	9
2.9	Security Requirements	10
2.10	Accessibility And Language Requirements	11
2.11	Notification Requirements	12
Chapter 3	**LITERATURE REVIEW**	13
3.1	Paper 1 – Web-Controlled Digital Notice Board	13
3.2	Paper 2 – Smart Digital Notice Board	13
3.3	Paper 3 – Web-Based College Notice Board	13
3.4	Paper 4 – Raspberry Pi Digital Notice Board	13
3.5	Paper 5 – AI Smart Notice Board	13
3.6	Paper 6 – Cloud Notice Board	14
3.7	Paper 7 – Android Notice Board	14
3.8	Paper 8 – Role-Based Access Control Models	14
3.9	Paper 9 – TextRank for Text Processing	14
3.10	Paper 10 – No Language Left Behind	14
Chapter 4	**DIAGRAMS**	16
4.1	Data Flow Diagram – Level 0	16

4.1.1	Data Flow Diagram – Level 1 Administrator	16
4.1.2	Data Flow Diagram – Level 1 Student	17
4.2	Use Case Diagram	18
4.3	Class Diagram	19
4.4	State Diagram	20
4.5	Activity Diagram	21
4.6	Sequence Diagram	22
4.8	System Architecture	23
Chapter 5	**METHODOLOGY**	24
5.1	Module Description – Authentication	24
5.2	Module Description – Notice Management	24
5.3	Module Description – Student Board	24
5.4	Module Description – Ai Assistance	24
5.5	Module Description – Translation	25
5.6	Module Description – Notifications	25
5.7	Algorithms And Workflow	25
5.8	System Architecture And Case Study	26
5.9	Development And Validation Workflow	30
5.10	Authentication Design	31
5.11	Department Targeting	32
5.12	Priority Scoring	33
5.13	Summarisation Procedure	34
5.14	Translation Workflow	35
5.15	Notification Outbox	36
5.16	Expiry And Archival	37
5.17	Private Attachment Handling	38
5.18	Case Study – Examination Timetable	39
Chapter 6	**IMPLEMENTATION DETAILS**	40
6.1	Implementation Procedure	40
6.2	Database And Storage	40
6.3	Testing Strategy	40
6.4	Validation And Limitations	41
6.9	Database Integrity	41
6.10	Notice Publication Transaction	43
6.11	Student Query Enforcement	44
6.12	Test Environment	45

6.13	Authentication Test Matrix	46
6.14	Notice Lifecycle Test Matrix	47
6.15	Access Control Test Matrix	48
6.16	Language And Assistant Test Matrix	49
6.17	Notification And Email Test Matrix	50
Chapter 7	**ADVANTAGES AND DISADVANTAGES**	51
7.1	Advantages	51
7.2	Disadvantages	51
Chapter 8	**RESULT AND DISCUSSION**	52
8.1	Results	52
8.2	Discussion	52
8.3	Role-Based Functional Analysis	53
8.4	Notification Responsiveness Discussion	54
8.5	Multilingual Access Discussion	55
8.6	Security And Privacy Discussion	56
Chapter 9	**CONCLUSION**	57
9.1	Summary	57
9.2	Limitations	57
9.3	Future Scope	57
9.4	Applications	58
9.5	Implementation Boundaries And Next Steps	59
Chapter 10	**APPENDICES**	60
	Sample Source – Role-Constrained Notice Access	60
Chapter 11	REFERENCES	62
11.1	Implementation References	62

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Functional Requirements	6
Table 2.2	Non-functional Requirements	6
Table 3.1	Comparative Review of Ten Related Studies	15
Table 5.1	Technology Components	26
Table 6.1	Testing and Validation	41

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 4.1	Data Flow Diagram – Level 0	16
Figure 4.2	Data Flow Diagram – Level 1 Administrator	16
Figure 4.3	Data Flow Diagram – Level 1 Student	17
Figure 4.4	Use Case Diagram	18
Figure 4.5	Class Diagram	19
Figure 4.6	State Diagram	20
Figure 4.7	Activity Diagram	21
Figure 4.8	Sequence Diagram	22
Figure 4.9	System Architecture	23
Figure 5.1	Student Notice Board	26
Figure 5.2	Administrator Dashboard	27
Figure 5.3	Mobile Notice Board	27
Figure 5.4	Student Filterin	28
Figure 5.5	Student Notice Translation	28
Figure 5.6	Assistant and Translation	29

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
Django	Python Web Framework
ER	Entity Relationship
HTML	Hypertext Markup Language
NLP	Natural Language Processing
PWA	Progressive Web Application
RBAC	Role-Based Access Control
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Physical notice boards are familiar to academic institutions, yet their information remains confined to a location and to the moment a reader checks it. Students on leave or away from campus may miss changes. General-purpose chats enable convenient sharing but mix official and informal posts without reliable categories, history or expiry control. CollegeNotice offers a dedicated system for publishing and retrieving institutional notices through authenticated web accounts.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

1.2 MOTIVATION

A student should be able to distinguish an examination deadline from an ordinary event invitation, find a notice issued last week and receive an alert for a newly published announcement. Administrators need to avoid repeating announcements across groups and manually removing out-of-date posters. These practical communication issues motivated the project.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

1.3 SCOPE AND OBJECTIVES

The scope covers department-aware notice publication, student access, authenticated attachments, searching, bookmarks, alerts, summaries, priority hints, Malayalam translation, an administrator dashboard and basic reporting. The primary objective is a consistent and accessible notice lifecycle from preparation through archival.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

1.4 PROBLEM STATEMENT

To develop an authenticated and accessible institutional notice system that reduces missed announcements, irrelevant information and manual administration through centralized publication, targeted access and automated lifecycle management.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

1.5 INSTITUTIONAL COMMUNICATION CONTEXT

Educational announcements need a reliable publication channel because examination schedules, application dates and departmental events may change rapidly. A departmental notice must be discoverable without requiring a student to be physically present near a paper notice board. CollegeNotice therefore models notices as records with authorship, publication boundaries and searchable classification rather than as static posters.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a

reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

1.6 STAKEHOLDER RESPONSIBILITIES

Students require a clear view of relevant active information and personal control over bookmarks and alert preferences. Department administrators need permission-scoped editing tools, and campus-wide publishers need higher privileges. Keeping these responsibilities separate helps prevent a department administrator from changing an unrelated department's notices.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

1.7 OBJECTIVE TRACEABILITY

The project objectives are measurable in terms of software behaviour rather than unsupported claims about learning outcomes. Notice creation, filtering, expiration, bookmarking, source-based question answering and language presentation can each be tested with a defined input and expected response.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 2
REQUIREMENTS AND ANALYSIS

2.1 EXISTING SYSTEM AND LIMITATIONS

Traditional campus notice boards require an individual to be physically present. Information shared through instant messages lacks an authoritative history, commonly gets buried and may be forwarded out of context. Neither channel consistently distinguishes student departments or automatically hides notices after expiry. The lack of a structured searchable archive also makes earlier instructions difficult to recover.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

2.2 SOLUTION OVERVIEW

The proposed system models department, category, user, notice, bookmark, notification, delivery and activity records. An administrator posts a notice through an authenticated interface. The backend validates department permissions and attachment properties, derives summary and priority metadata, creates student notifications and queues email jobs. Students read published and unexpired notices using search and filters.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

2.3 FUNCTIONAL REQUIREMENTS

Core operations include account registration and login; posting, editing, archiving and deleting notices; department and category assignment; publication and expiry dates; keyword and priority filters; bookmark management; notification tracking; local translation; grounded assistant responses; and an administrator reporting dashboard.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

Table 2.1: Functional Requirements

ID	Function	Expected System Behaviour
FR1	User authentication	Students and authorised staff can sign in; server routes enforce role and department permissions.
FR2	Notice publication	Permitted staff create, edit, publish, archive and restore notices.
FR3	Notice classification	Notices may be assigned a department, category and publication/expiry dates.
FR4	Search and filtering	Students retrieve active notices using keywords, date, department, category and priority filters.
FR5	Personal bookmarks	Students save notices and remove saved items from their bookmarks.
FR6	Notifications	Relevant in-app alerts are stored and can be marked as read; email jobs are tracked.
FR7	Text assistance	Local summarisation and notice-grounded question answering provide source-linked text.
FR8	Translation and reporting	English–Malayalam translation, administrator dashboard and CSV reporting are supported.

2.4 NON-FUNCTIONAL REQUIREMENTS

Security requires authorization on every private operation, CSRF protection for browser writes, protected attachment delivery and password hashing. Usability requires responsive screens and accessible labels. Maintainability requires clear modules and repeatable database migrations. Reliability requires persistent email delivery states and safe recovery of unprocessed tasks.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

Table 2.2: Non-functional Requirements

Quality Attribute	Requirement	Verification Approach
Security	Authenticated access, server-side authorisation, CSRF protection and protected attachments.	Permission-denial, login and attachment access tests.
Usability	Responsive screens and understandable notice details across common device widths.	Browser walkthrough of student and staff screens.
Reliability	Expiry filters remain correct even if a scheduled job has not run; email job states persist.	Lifecycle and delivery-state tests.
Maintainability	Django modules, database migrations and reusable service logic support modification.	Review of project organisation and migrations.
Accessibility	Readable labels and optional Malayalam assistance while retaining original notice text.	Manual review of interfaces and sample translations.
Portability	SQLite supports the local demonstration; PostgreSQL, Celery and Redis are separately configured.	Local startup and configuration inspection; distributed profile requires deployment tests.

2.5 FEASIBILITY STUDY

Python and Django offer documented components for authentication, routing and relational modelling. Browser-standard HTML, CSS and JavaScript avoid a separate frontend build. Open-source PostgreSQL, Redis and Celery are suitable for background tasks when infrastructure is

available. A local SQLite demonstration remains useful on ordinary Windows machines, with functionality distinguished from a distributed deployment.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

2.6 RISKS AND CONSTRAINTS

Email delivery depends on external SMTP credentials; local demo messages do not reach live inboxes. Translation needs an additional language model and runtime installation. Public hosting requires HTTPS, site-specific secrets, account verification and monitoring. These are deployment constraints rather than functionality already demonstrated.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

2.7 USER AND ROLE REQUIREMENTS

The two main roles are students and authorised publishing staff. The application must resolve a signed-in identity before returning private content. A student can read active permitted notices and save bookmarks; an administrator can manage notices within the granted department scope. Campus-wide publication is restricted to superusers.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

2.8 NOTICE VALIDATION

A notice is associated with category, publication and expiration dates, and may target one department or all departments. Input validation prevents inconsistent dates and rejects unauthorised updates. The application must apply visibility rules to querysets and resource retrieval endpoints, not only to the user interface.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

2.9 SECURITY REQUIREMENTS

Passwords must be stored with a strong password hashing mechanism. Browser login relies on Django session cookies with CSRF protection; external API integrations can use short-lived JWT access tokens. Public links must not expose private uploaded files directly.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

2.10 ACCESSIBILITY AND LANGUAGE REQUIREMENTS

layout should respond to mobile and desktop widths. English and Malayalam language services are implemented locally; if language conversion is unavailable for a particular notice, the original must remain intact instead of being silently replaced with an unreliable translation.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

2.11 NOTIFICATION REQUIREMENTS

Alerts are stored as user-specific records. The browser refreshes unread notification status periodically when visible, which supports near-real-time communication but is different from WebSocket delivery. Email jobs remain recorded so delivery can be attempted or retried when a configured service is available.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 3
LITERATURE REVIEW

The following ten entries compare digital-notice-board approaches with related security and language-processing research. The first seven entries reproduce the studies described in the supplied presentation. Full publication details for entries 2–7 could not be independently confirmed; their algorithms and findings are recorded as presentation claims rather than verified experimental results.

3.1 PAPER 1 – Web-Controlled Centralized Digital Notice Board for Smart Institutions (2025)

Ninawe, Sarangpure, Sahare, Shende and Yadhav (2025) present a web-controlled notice board integrating event management and real-time updates across devices. The publisher describes a responsive online notice and event platform. The earlier project presentation assigns NLP as its algorithm and summarizes the finding as reduced manual work; the accessible publisher abstract does not independently establish an NLP implementation or a measured efficiency result. This work is relevant to centralised campus notice publication.

3.2 PAPER 2 – Smart Digital Notice Board (2023)

The supplied CollegeNotice presentation attributes this 2023 smart notice board study to S. Kumar and describes an MQTT-based approach for remote notice delivery with cloud and mobile alerts. Its reported advantage is faster dissemination and less dependence on printed notices. A complete bibliographic record was not supplied or independently confirmed; these details are therefore treated as a presentation summary rather than a verified experimental finding.

3.3 PAPER 3 – Web-Based College Notice Board (2022)

The project presentation lists A. Sharma (2022) for a web-based college notice board using role-based login and departmental announcement access. The stated benefit is more controlled publishing and a clearer experience for users. CollegeNotice similarly applies role checks on the server instead of relying on hidden interface buttons. The supplied author and journal details have not been independently verified, and no satisfaction-study measurements are available.

3.4 PAPER 4 – Digital Notice Board Raspberry Pi (2021)

As summarised in the supplied slides, M. Reddy (2021) describes an electronic notice board built around Raspberry Pi display control and wireless notice updates. This is a display-oriented alternative to paper boards. Unlike the browser-oriented CollegeNotice system, it emphasises a shared physical display endpoint. The precise IEEE publication and quantitative cost or power evaluation were not independently established from the supplied bibliographic information.

3.5 PAPER 5 – AI Smart Notice Board (2024)

The supplied literature slide includes J. Lee (2024) under AI-assisted notice boards and associates the study with NLP for summarising and translating published messages. These capabilities are related to CollegeNotice's local language helpers. The Springer publication details, method design and

outcomes are not independently verified from the presentation; the stated improvements are not reproduced here as measured research results.

3.6 PAPER 6 – Cloud Notice Board (2023)

The project presentation describes P. Patel (2023) as a cloud-notice-board proposal emphasising synchronised storage, authentication and backup. Such work is relevant to a future deployment of CollegeNotice beyond its local SQLite demonstration. The supplied entry does not contain enough publication metadata to verify the claimed Elsevier source or performance measures, so availability and scalability are recorded only as reported benefits in the slides.

3.7 PAPER 7 – Android Notice Board (2022)

According to the literature summary in the provided presentation, R. Gupta (2022) used Firebase for Android notice delivery and push notifications. The proposal highlights convenient mobile access and proactive delivery. CollegeNotice instead provides a responsive web board with polling-based notifications in its local implementation. The publication details and any engagement improvement remain unverified because no identifying DOI, journal issue or full text was provided.

3.8 PAPER 8 – Role-Based Access Control Models (1996)

Sandhu, Coyne, Feinstein and Youman (1996) describe a family of role-based access control models in which users are assigned roles and permissions are associated with those roles. The models simplify permission administration and form a useful security foundation for CollegeNotice, where only authorised department staff may publish or edit scoped notices. This study concerns access-control design rather than a specific digital notice-board implementation.

3.9 PAPER 9 – TextRank: Bringing Order into Text (2004)

Mihalcea and Tarau (2004) introduce TextRank, an unsupervised graph-based approach to ranking textual units for keyword and sentence extraction. The published evaluation reports encouraging results on established text-processing datasets. CollegeNotice uses local frequency- and deadline-cue summarisation rather than implementing TextRank; this paper provides related theoretical context, not a claim about the project's actual algorithm.

3.10 PAPER 10 – No Language Left Behind: Scaling Human-Centered Machine Translation (2022)

Costa-jussà and colleagues (2022), writing as the No Language Left Behind research team, present methods and multilingual models intended to improve translation coverage for low-resource languages. The NLLB family motivates CollegeNotice's local English–Malayalam translation capability. The paper's published benchmarks should not be interpreted as CollegeNotice translation accuracy; project-specific assessment is still necessary.

Table 3.1: Comparative Review of Ten Related Studies

Title & Year	Authors	Algorithm	Key Features	Findings
Web-Controlled Centralized Digital Notice Board for Smart Institutions (2025)	Madhuri Ninawe, Pooja Sarangpure et al.	NLP (claimed in slides; not verified)	Real-time notice and event updates, event management, responsive web access	Reduced manual administration (presentation summary)
Smart Digital Notice Board (2023)	S. Kumar (IEEE, as supplied)	MQTT	Remote publication, cloud connectivity and mobile notifications	Instant delivery and reduced paper use (as supplied)
Web-Based College Notice Board (2022)	A. Sharma (IJCA, as supplied)	Role-Based Access Control (RBAC)	Role-specific login and department-targeted notices	Secure access and improved user satisfaction (as supplied)
Digital Notice Board Raspberry Pi (2021)	M. Reddy (IEEE, as supplied)	Display control	Raspberry Pi, wireless notices and electronic display	Low-cost and low-power display (as supplied)
AI Smart Notice Board (2024)	J. Lee (Springer, as supplied)	Natural Language Processing (NLP)	Notice summarisation and multilingual presentation	Improved readability and accessibility (as supplied)
Cloud Notice Board (2023)	P. Patel (Elsevier, as supplied)	Cloud synchronisation	Cloud database, secure authentication and backup	Availability and scalability (as supplied)
Android Notice Board (2022)	R. Gupta (IJERT, as supplied)	Firebase	Mobile notice viewing and push notifications	Higher engagement (as supplied)
Role-Based Access Control Models (1996)	R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman	RBAC model family	Roles, permissions and simplified security administration	Reference models organise permission assignment
TextRank: Bringing Order into Text (2004)	Rada Mihalcea and Paul Tarau	Graph-based TextRank ranking	Unsupervised keyword and sentence extraction	Competitive extraction on published evaluation tasks
No Language Left Behind: Scaling Human-Centered Machine Translation (2022)	NLLB Team / M. R. Costa-jussà et al.	Multilingual neural machine translation	Translation for many languages, including lower-resource language pairs	Improved translation coverage; quality depends on language and domain

Notes: Entries 2–7 are retained exactly in substance from the supplied presentation and have incomplete/unverified bibliographic details. Reported advantages are qualitative and must not be treated as CollegeNotice test metrics. TextRank, RBAC models and NLLB are related research; their specific algorithms are not claimed as wholly implemented within CollegeNotice.

CHAPTER 4
DIAGRAMS

4.1 DATA FLOW DIAGRAM – LEVEL 0

Context-level data flow illustrates the boundary between students, administrators and the digital notice board application.

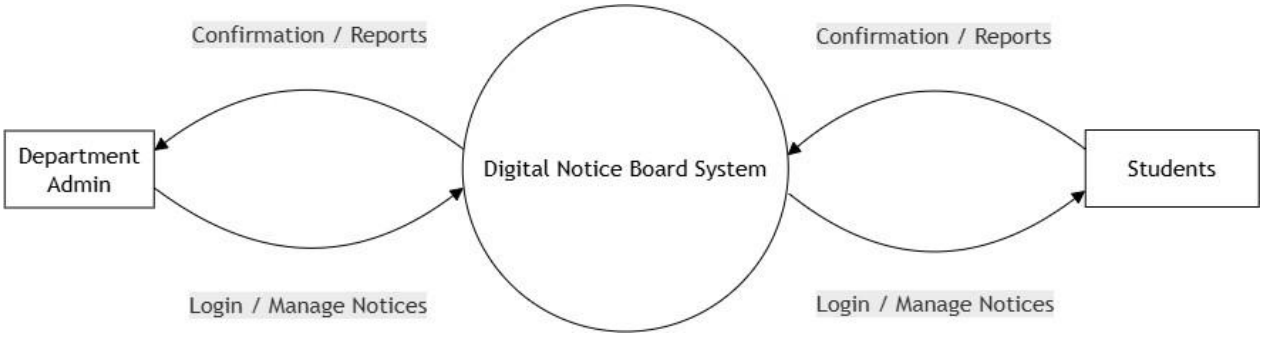


Figure 4.1: Data Flow Diagram – Level 0

4.1.1 DATA FLOW DIAGRAM – LEVEL 1 ADMINISTRATOR

The administrator flow separates publication and maintenance responsibilities, showing the inputs that create or modify notice records.

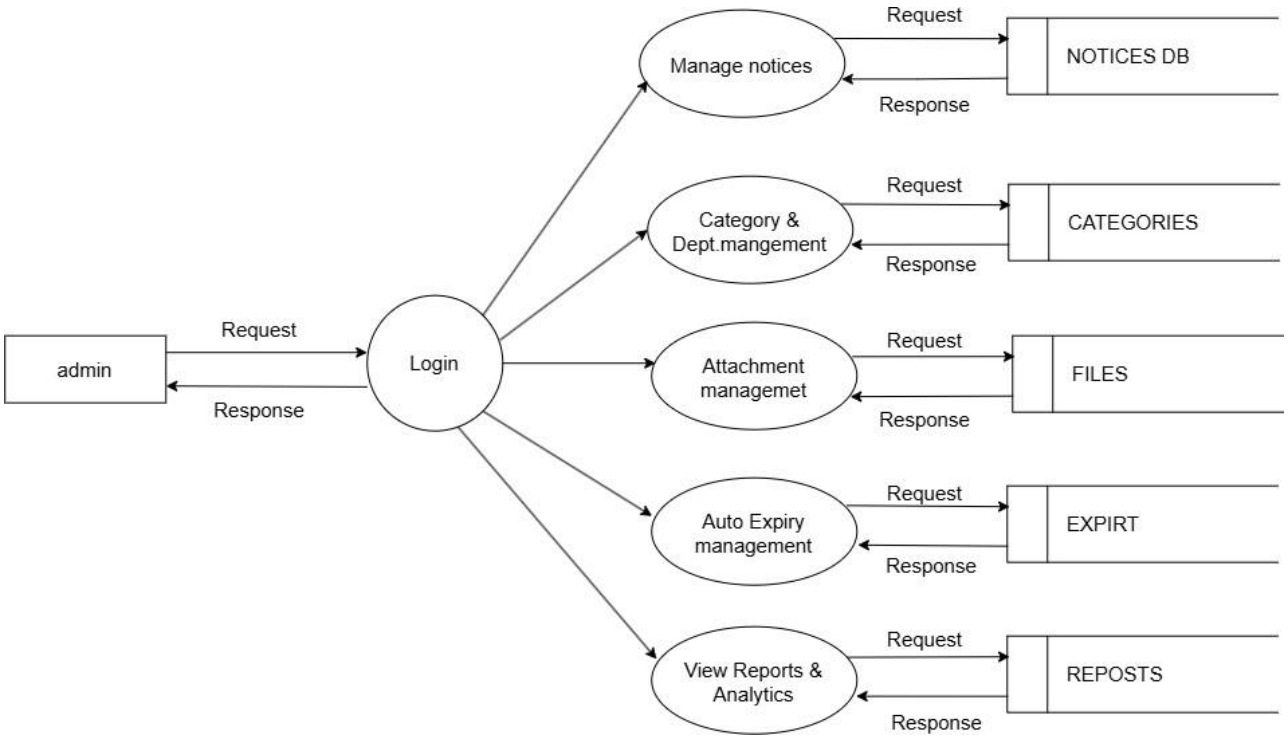


Figure 4.2: Data Flow Diagram – Level 1 Administrator

4.1.2 DATA FLOW DIAGRAM – LEVEL 1 STUDENT

The student flow represents notice viewing, search and filtering, personal bookmarks and notification retrieval.

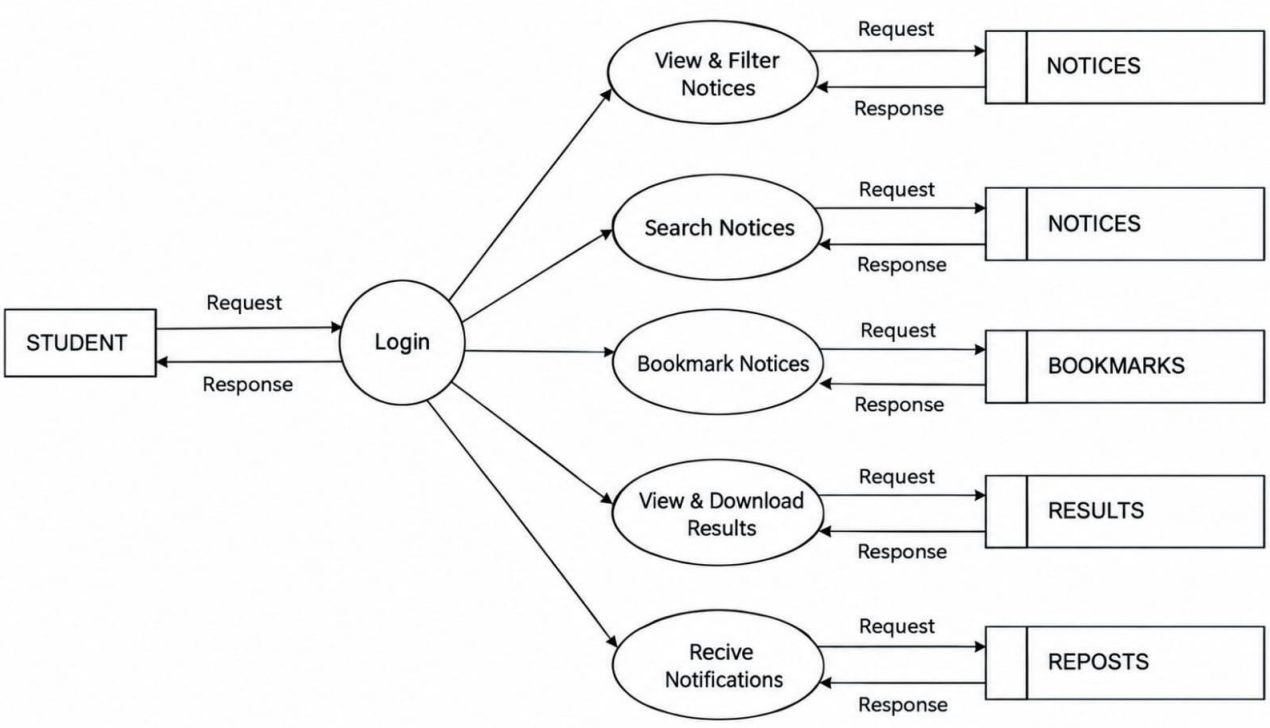


Figure 4.3: Data Flow Diagram – Level 1 Student

4.2 USE CASE DIAGRAM

The use case diagram describes the allowed interactions for students and institutional publishers.

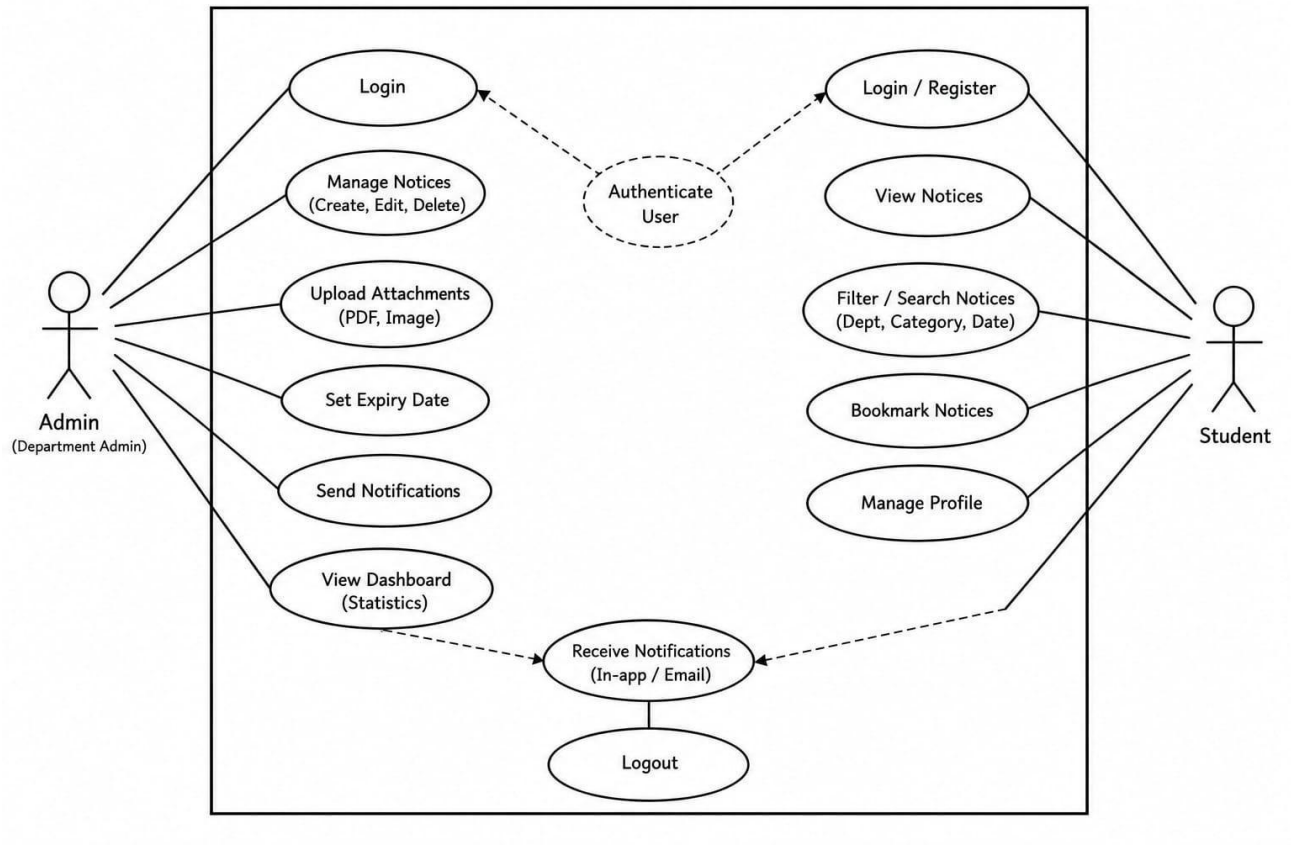


Figure 4.4: Use Case Diagram

4.3 CLASS DIAGRAM

A class view can be derived from the implemented Django models: User, Department, Category, Notice, Bookmark, Notification, Email Delivery and Activity. Their relations and constraints are documented in the architecture and database chapters; the submitted PPT does not provide a separate UML class diagram.

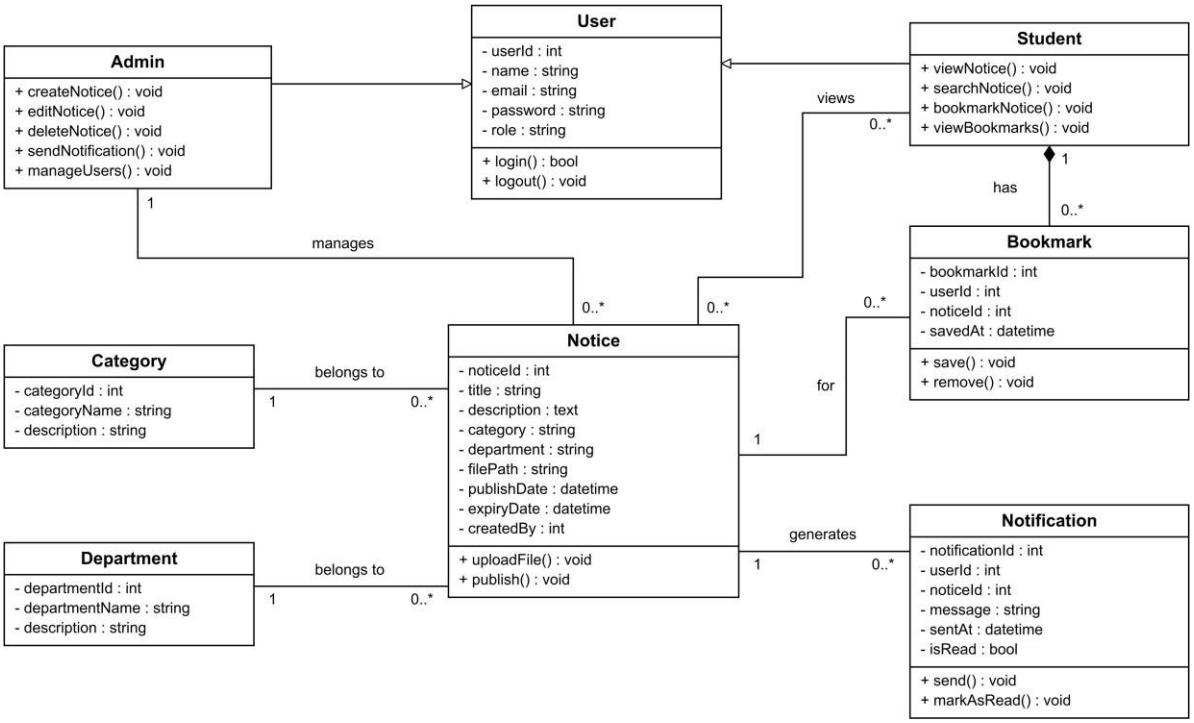


Figure 4.5: Class Diagram

4.4 STATE DIAGRAM

Notice visibility changes with publication and expiry dates and archive actions. The PPT does not contain a separate UML state diagram; a textual transition description is included here to avoid presenting an invented image as supplied evidence.

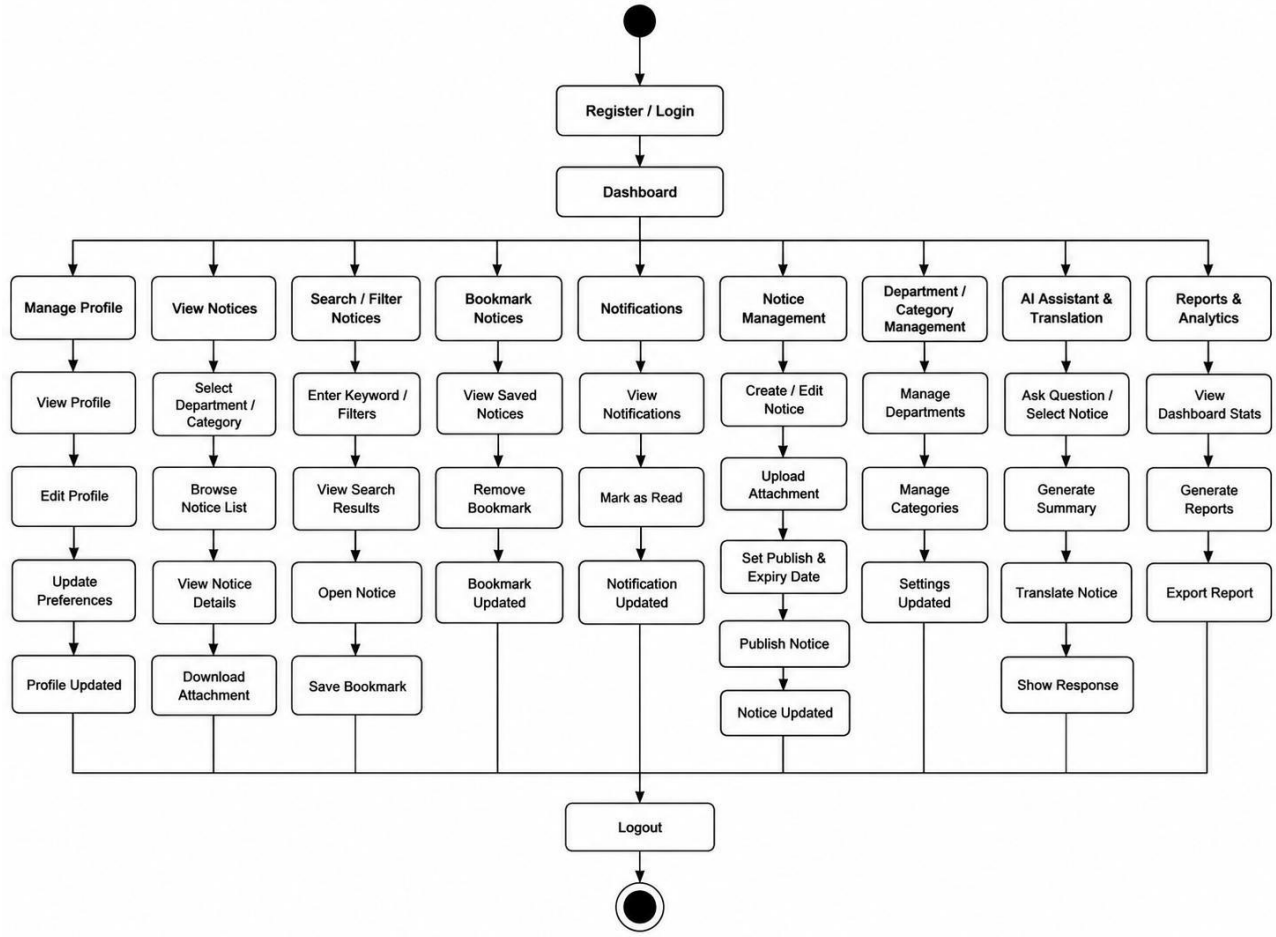


Figure 4.6: State Diagram

4.5 ACTIVITY DIAGRAM

An administrator authenticates, enters validated notice details, selects a department and publication interval, saves the record, and triggers alerts. The student subsequently retrieves authorised active notices. No separately drawn activity diagram was included in the submitted PPT.

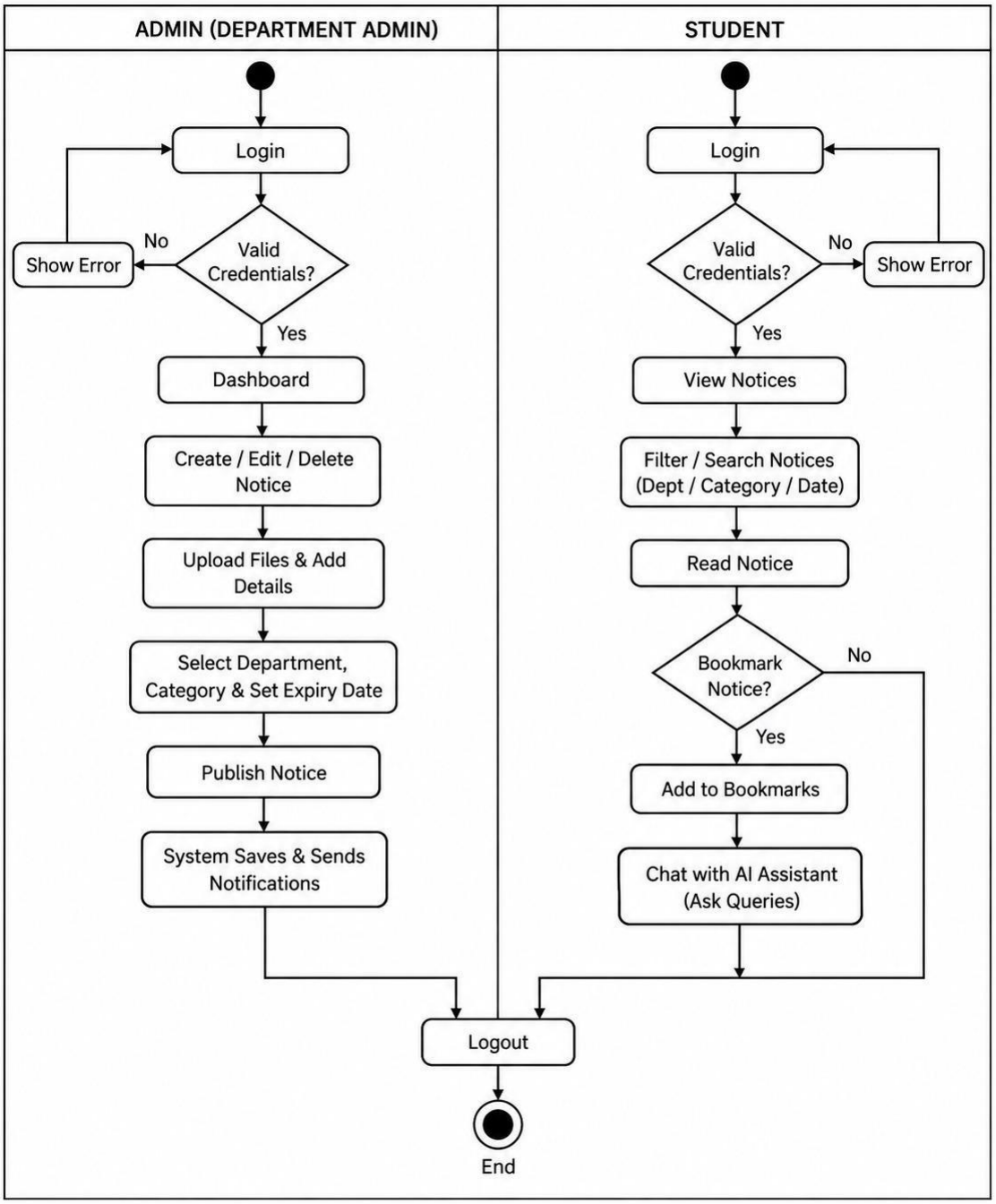


Figure 4.7: Activity Diagram

4.6 SEQUENCE DIAGRAM

The sequence diagram depicts interactions between a user interface, service layer and stored notice data during a typical request.

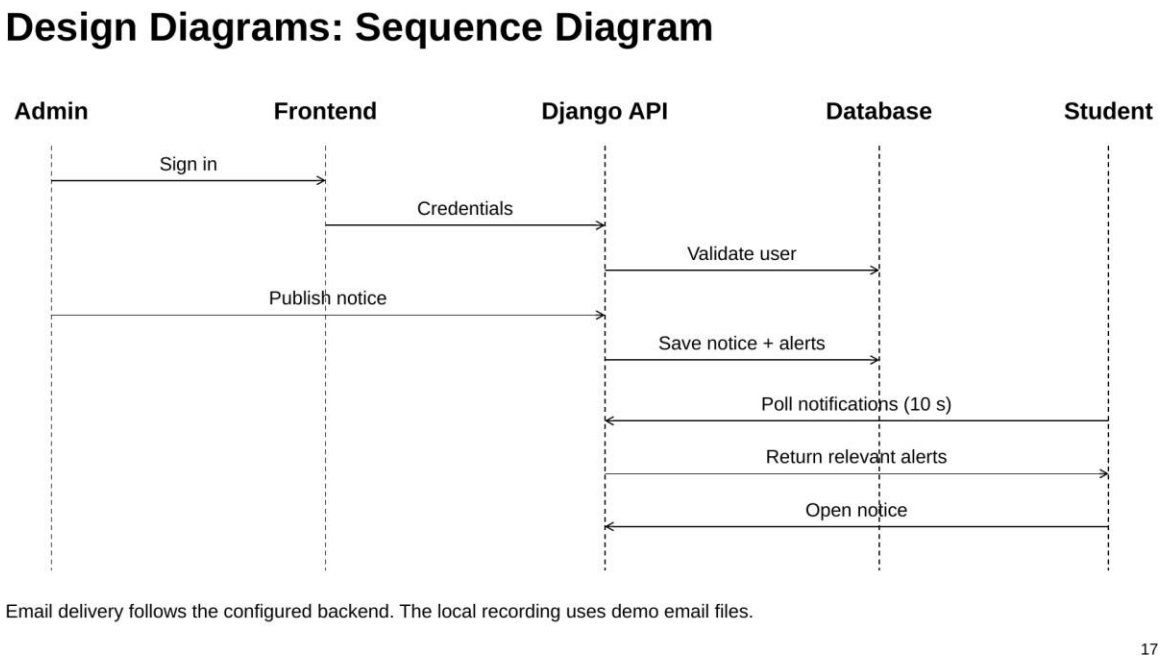


Figure 4.8: Sequence Diagram

4.8 SYSTEM ARCHITECTURE

The proposed architecture places Django APIs between browser requests, relational persistence, local NLP services and background workers.

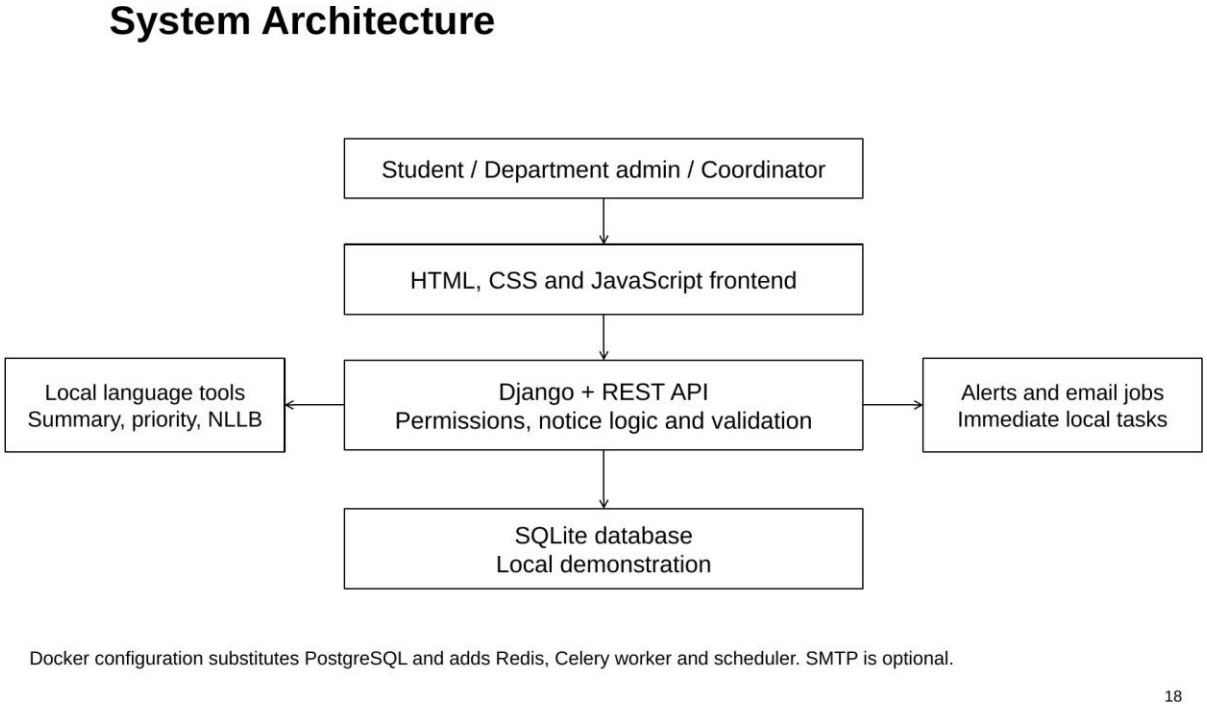


Figure 4.9: System Architecture

CHAPTER 5
METHODOLOGY

5.1 MODULE DESCRIPTION – AUTHENTICATION

The user model extends the Django user abstraction. Browser access uses HttpOnly sessions and CSRF validation. JWT access and refresh endpoints are available to API clients, with token rotation and revocation support. Department assignment controls the scope of ordinary administrator publishing. Campus-wide publication is reserved for the coordinator role.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.2 MODULE DESCRIPTION – NOTICE MANAGEMENT

The notice manager provides create, retrieve, update, delete, archive and restore operations. Each record carries a department target, category, content, author and date metadata. Attachments have explicit file-type and size validation and are served after authorization rather than being exposed as unrestricted static files.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.3 MODULE DESCRIPTION – STUDENT BOARD

The student board supports text search, date filters, category and department filtering, priority-only views and bookmarks. Notice lists are paginated, and expired content is excluded from current public queries. The responsive interface adapts to smaller screens and uses a Progressive Web App manifest.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.4 MODULE DESCRIPTION – AI ASSISTANCE

An extractive summarizer generates concise text from an announcement rather than creating unsupported details. Priority detection uses the words and patterns of the notice. The assistant searches available notice information and returns linked sources, helping the student verify the original announcement. This is different from unrestricted free-form conversation.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.5 MODULE DESCRIPTION – TRANSLATION

English–Malayalam translation uses an optional locally installed multilingual model. The installer downloads a substantial amount of model data, so translation cannot be assumed available in an untouched ZIP. If unavailable, the application preserves the original text and surfaces status rather than inventing a translation.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.6 MODULE DESCRIPTION – NOTIFICATIONS

In-app notifications are created for relevant students, with unread counts presented by a ten-second refresh cycle. Email jobs retain state in the database and can be processed by background workers in the full configuration. The local demo saves sample email files without actually sending them.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.7 ALGORITHMS AND WORKFLOW

The main algorithms include validation of role and department constraints, keyword and date filtering, extractive sentence selection for summaries, rule-based priority tagging, retrieval-based notice assistance and persistent state transitions for notification delivery. Their purpose is to make information relevant, readable and trustworthy.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

Table 5.1: Technology Components

Layer	Component	Role in CollegeNotice
Backend	Python and Django	Handles routes, models, notice processing, business rules and authentication.
API	Django REST Framework	Exposes role-constrained notice and account operations.
Frontend	HTML, CSS and JavaScript	Provides the responsive notice board, staff views, filtering and notification polling.
Local database	SQLite	Stores users, notices, categories, bookmarks and related records during demonstration.
Optional deployment DB	PostgreSQL	Relational persistence in the Docker configuration.
Background processing	Celery and Redis (configured)	Provide a distributed worker and broker option; immediate task execution is used locally.
Language assistance	Local rules, NLLB, CTranslate2 and SentencePiece	Supports priority cues, summaries and optional local English–Malayalam translation.
Browser security	Django sessions/CSRF; JWT for API clients	Controls signed-in browser requests and separate API-token authentication.

5.8 SYSTEM ARCHITECTURE AND CASE STUDY

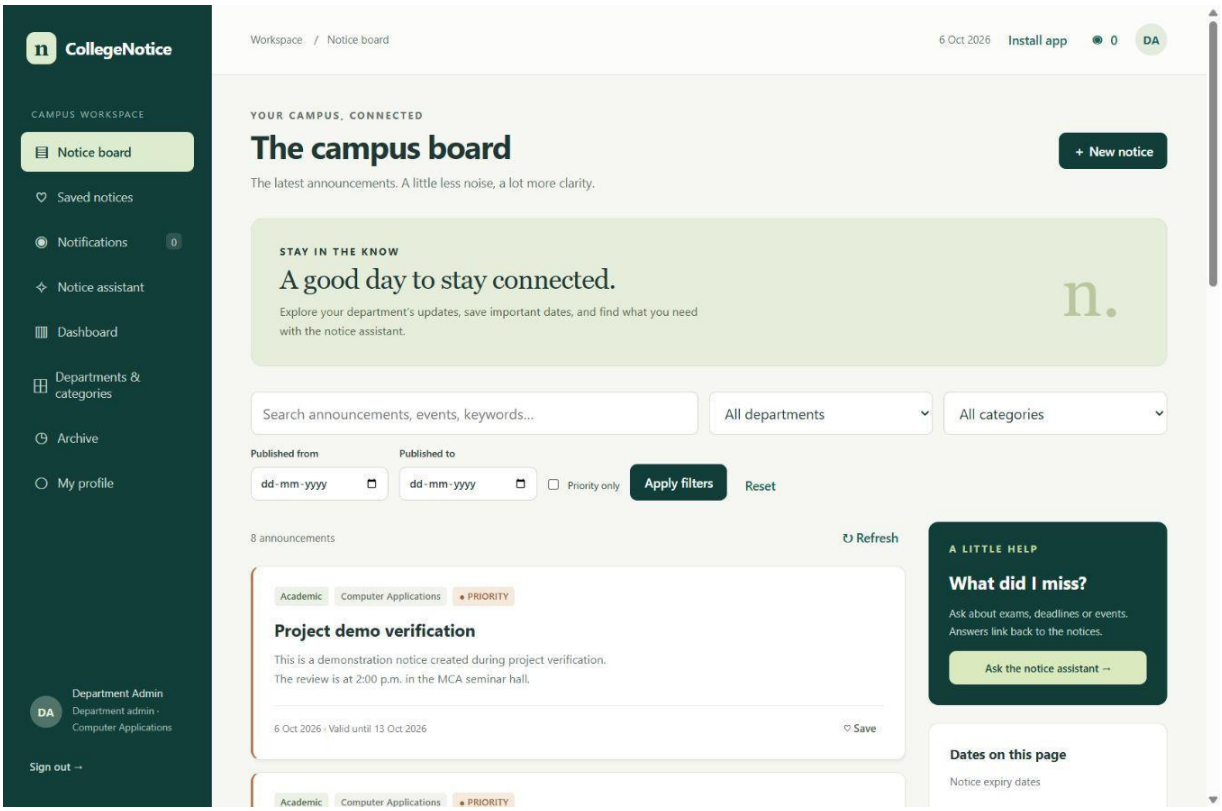


Figure 5.1: Student Notice Board

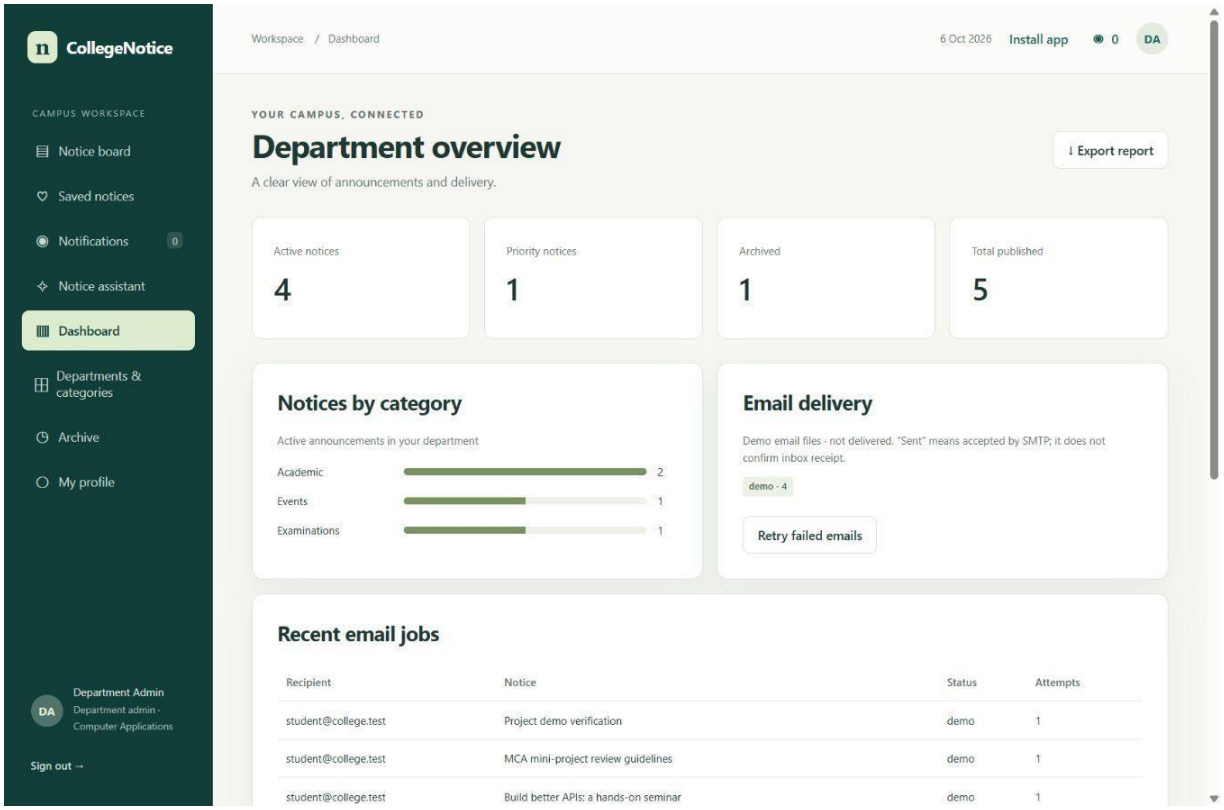


Figure 5.2: Administrator Dashboard

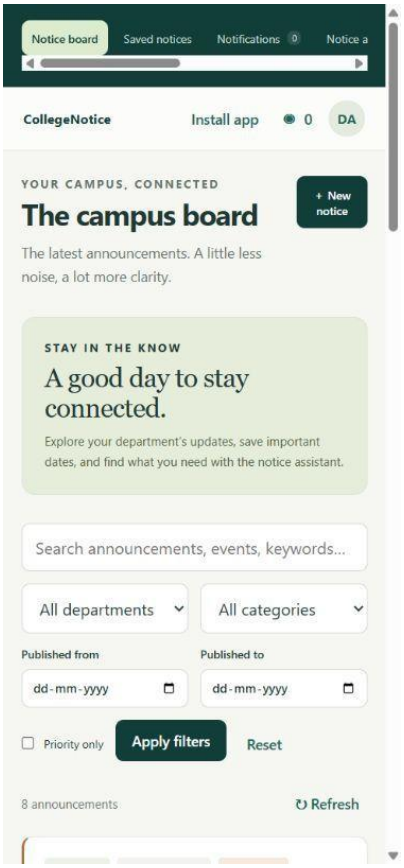


Figure 5.3: Mobile Notice Board

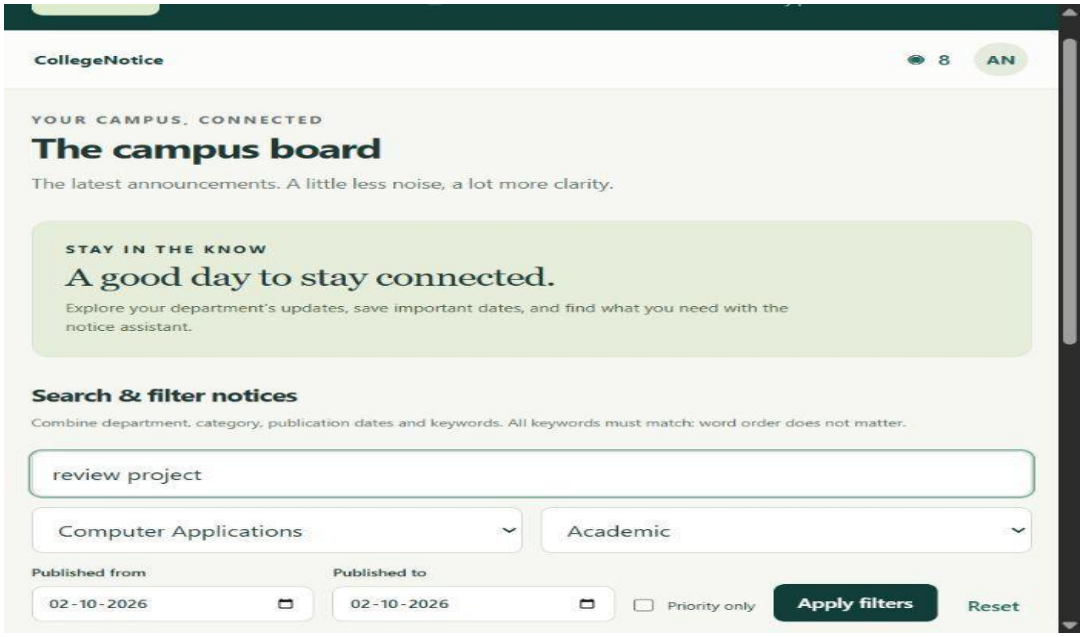


Figure 5.4: Student Filterin

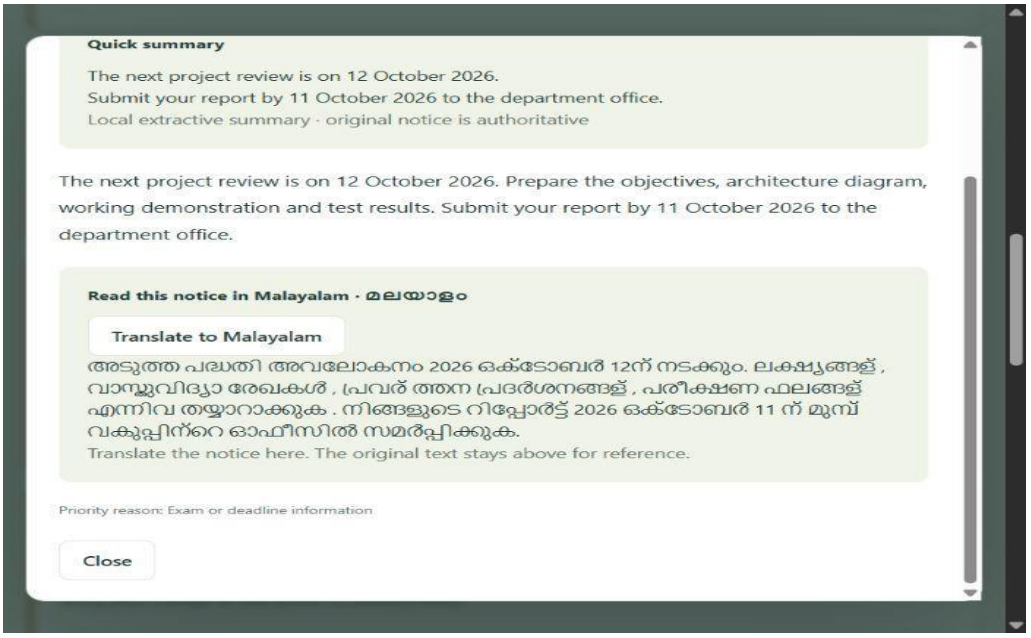


Figure 5.5: Student Notice Translation

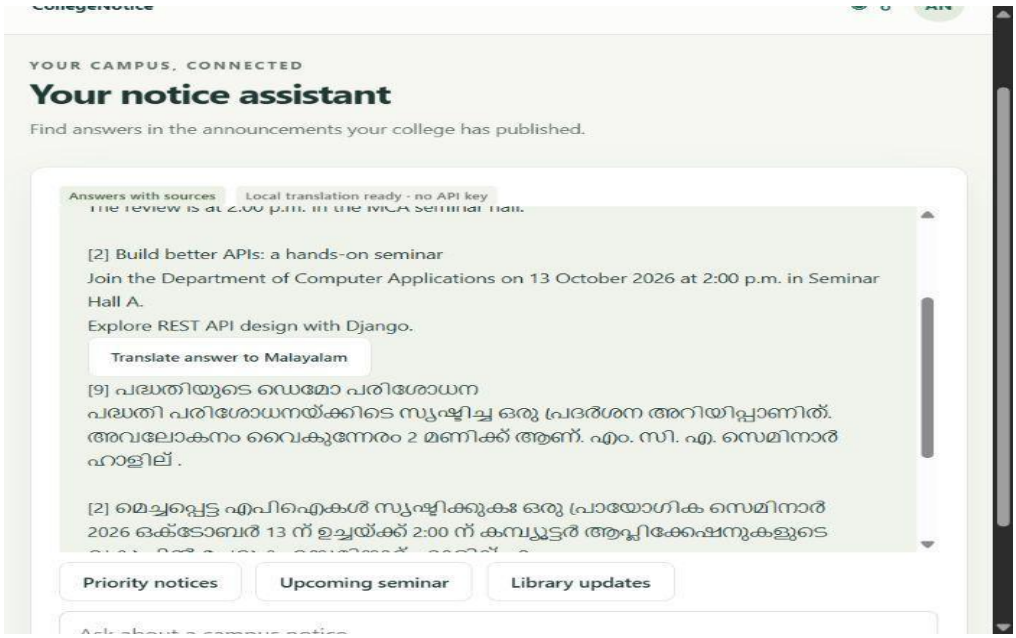


Figure 5.6: Assistant and Translation

Consider an administrator posting an examination schedule for the MCA department. The system checks permissions, stores the document and publication dates, derives an urgency hint, and creates notifications for eligible students. A student uses the priority filter, reads the original PDF and bookmarks the notice. In the local demonstration the corresponding email job is recorded without claiming actual inbox delivery.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

5.9 DEVELOPMENT AND VALIDATION WORKFLOW

Implementation proceeds from domain models and permission checks to API serializers, page templates, and then supplementary language and alert services. This sequence reduces the risk that visually complete pages conceal inconsistent authorisation. Validation routines are applied before publication and repeated in regression tests.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.10 AUTHENTICATION DESIGN

The browser authenticates using Django sessions and protected cookie credentials. Unsafe state-changing requests require CSRF validation. Separate JWT token endpoints are intended for API clients, and those tokens are not stored in browser local storage. This separation reduces avoidable token exposure in the regular web application.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.11 DEPARTMENT TARGETING

The target department determines which students should see a notice and which staff may modify it. A null department is treated as campus-wide and requires higher publication permission. A server-side filter combines the signed-in student's department with campus notices and active publication dates.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.12 PRIORITY SCORING

Priority detection is implemented with local patterns and deadline emphasis, not a model trained on an external benchmark dataset. Priority output is useful as a visual cue for examination or closing-date notices; however, administrators and students should still read the complete source information to assess consequences.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the

5.13 SUMMARISATION PROCEDURE

The summariser derives a shortened representation from the source text. Token frequency and relevant date cues can elevate informative phrases, but extractive shortening does not verify the substance of an announcement. Therefore each displayed summary is supplementary and the unabridged notice stays available.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.14 TRANSLATION WORKFLOW

The app first stores the original notice and then derives translated presentation text with a local NLLB service when installed. Model unavailability must be visible as a supported fallback rather than producing a false translation. Preserving the source allows bilingual comparison and correction.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.15 NOTIFICATION OUTBOX

Publishing a notice creates notification records and persistent email jobs in the same database transaction. Background delivery starts only after the transaction commits. With the full stack, Redis and Celery process jobs; local demonstration mode executes tasks immediately without representing actual SMTP delivery as completed.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.16 EXPIRY AND ARCHIVAL

The notice query excludes expired entries regardless of whether the scheduled archive job has executed. A separate maintenance routine can persist an archived state for administration and cleanup. This distinction provides correct student visibility during worker interruptions and makes maintenance a recoverable operation.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.17 PRIVATE ATTACHMENT HANDLING

Uploaded attachments are recorded with the notice and stored under generated names. Student downloads flow through authenticated access checks instead of publicly browsable media paths. A file record is therefore governed by the same visibility and role rules that protect the associated notice.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

5.18 CASE STUDY – EXAMINATION TIMETABLE

Consider an administrator publishing a revised examination timetable for a specific department. The admin chooses a category, target department, publication interval and attachment. Eligible students see the active notice after the next refresh; students elsewhere do not gain edit rights. This is an illustrative use case, not a measured production deployment.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 6
IMPLEMENTATION DETAILS

6.1 IMPLEMENTATION PROCEDURE

The code base separates Django project configuration from the board application and from the responsive static interface. Relational models define notices, students, categories, notifications and outbox records. Serializers enforce upload and role validation, views implement REST workflows, and scheduled tasks handle retries and archival. Database migration files preserve repeatable schema creation.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

6.2 DATABASE AND STORAGE

The demonstration configuration uses SQLite so the project starts without an external server. The presentation configuration is described by Docker Compose and uses PostgreSQL. Media attachments are protected, and stored file names are randomized. The project must be configured separately for encrypted backups and production storage.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

6.3 TESTING STRATEGY

Source-level tests cover permission enforcement, notice visibility, relevant filters, AI assistance and translation functions. Manual demonstration checks focus on role boundaries and page behaviour. The repository documents exactly which services were verified and which configurations still require integration testing.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

6.4 VALIDATION AND LIMITATIONS

Documentation states that Docker was unavailable on the validation computer, so distributed Celery and PostgreSQL execution was not fully integration-tested in that environment. The user interface and code should therefore not be described as evidence that a cloud deployment has been completed. This separation makes the report auditable.

The design decision has a direct operational consequence: information is presented in an explicit workflow, with a clear distinction between student-visible content, administrative capabilities and background services. The implementation notes and validation records describe the corresponding behaviour so that reviewers can inspect the source and reproduce the local demonstration.

Table 6.1: Testing and Validation

Test Area	Test Method	Validation Criterion / Reported Outcome
Authentication and roles	Automated session, token and permission checks.	Valid users authenticate; unauthorised writes are denied.
Notice lifecycle	API/unit tests for creation, publication, expiration and archive.	Published active notices display; expired or archived notices are filtered.
Search and bookmarks	Student browser walkthrough and data-level tests.	Matching notices appear and saved notices persist per user.
Notifications	In-app alert and background job-state tests.	Unread counts and stored delivery states remain consistent.
Language tools	Local summary, assistant-source and translation examples.	Answers retain links to notices; generated text is reviewed against source.
Integration boundary	Local SQLite/eager-task verification and Docker config review.	Distributed PostgreSQL/Redis/Celery operation requires a separate deployment test.

6.9 DATABASE INTEGRITY

Relational foreign keys bind notices to users, categories and optional departments. Database uniqueness constraints prevent a user from bookmarking or receiving identical notice-alert pairs repeatedly. Model constraints and serializer checks together prevent malformed lifespan values and category relationships.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.10 NOTICE PUBLICATION TRANSACTION

Publishing should be atomic: saving the notice, writing the activity record, and preparing in-app alerts and email jobs form a consistent unit. Starting external delivery only after commit prevents a worker from acting on data that may later be rolled back. Failure recovery depends on durable job status.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.11 STUDENT QUERY ENFORCEMENT

When a student requests a notice list, the server applies publication window, archive status and department access criteria before pagination. Search, priority and category filters operate on this allowed set. Applying permissions before filtering prevents accidental disclosure through counts or API detail routes.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.12 TEST ENVIRONMENT

The available repository documents local tests, browser checks and an optional full-stack runtime. Where a service was mocked, executed in eager mode, or configured but not deployed, the report records that distinction. A valid test result describes the environment and assertions, not an unverified production availability claim.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.13 AUTHENTICATION TEST MATRIX

Positive login tests check that a valid student receives session access. Negative tests should reject invalid passwords and attempts to call administrative endpoints. Session logout should revoke subsequent browser access; a separate API-token test checks bearer-token expiry and revocation characteristics.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.14 NOTICE LIFECYCLE TEST MATRIX

The tests should create a notice, update permitted fields, verify visibility, expire it and confirm it no longer appears to students. Archive and restore operations should respect administrator ownership, and category or department changes must not leave inconsistent references.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.15 ACCESS CONTROL TEST MATRIX

A student must not create, edit or delete notices, nor download an inaccessible attachment. A department administrator must not modify notices owned by another department. Superuser operations require their higher assigned privilege rather than relying on user interface positioning.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.16 LANGUAGE AND ASSISTANT TEST MATRIX

Translation samples should include common notice phrases, dates and departmental vocabulary. Where the local language model is not installed, tests should verify an explicit fallback. Assistant responses should link to authorised current notices and avoid treating unsupported answers as facts.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

6.17 NOTIFICATION AND EMAIL TEST MATRIX

A new active notice should create an appropriate user alert. The 10-second polling behaviour must be evaluated as delayed refresh rather than push notification. Email tests can use a test backend or mocked SMTP service, while live message delivery remains dependent on configured credentials.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 7
ADVANTAGES AND DISADVANTAGES

7.1 ADVANTAGES

The system provides a centralized platform for managing institutional announcements, ensuring effective and consistent communication. Department-based targeting helps deliver relevant information to specific users, reducing unnecessary notifications. Expiry controls keep outdated announcements from cluttering the interface, while bookmarking enables users to save important information for future reference. Local language support improves accessibility and helps users understand announcements more easily. The lightweight interface and offline-capable features support usability during temporary network interruptions. The modular technology stack also simplifies maintenance, supports future enhancements, and allows gradual deployment according to institutional requirements.

7.2 DISADVANTAGES

The system requires a suitable device and internet connectivity to access updated announcements. Limited network availability may delay access to new information, while periodic notification polling can result in slower updates compared to real-time push notifications. Local language models may increase storage consumption and processing requirements, particularly on devices with limited resources. The system also requires proper SMTP configuration for email notifications and reliable hosting infrastructure for continuous operation. Security implementation, database backups, performance optimization, and regular maintenance require additional administrative effort. These limitations can be reduced through infrastructure improvements, efficient notification mechanisms, and appropriate testing before production deployment.

CHAPTER 8
RESULT AND DISCUSSION

8.1 RESULTS

The application provides functional screens for the main notice board, department administration, mobile view, announcement filtering, and translation. It includes unit tests, workflow tests, and supporting documentation for validation and demonstration. These features demonstrate a working mini-project that supports centralized announcement management, user interaction, and improved accessibility.

The system allows administrators to publish and manage announcements while users can view relevant notices based on their department and preferences. The filtering and translation features make information easier to locate and understand. The responsive interface also supports convenient access across different screen sizes, improving usability for students and faculty members.

8.2 DISCUSSION

The project successfully integrates announcement management, role-based access control, search, notifications, and language assistance into a unified web application. These features improve information accessibility and simplify institutional communication. However, production-level performance, email delivery reliability, and high-load handling have not been experimentally evaluated. Further testing with actual institutional users is required to assess scalability, reliability, and overall effectiveness.

The implementation demonstrates how web technologies can reduce dependence on traditional notice boards and fragmented communication channels. The modular design also allows future improvements, including real-time notifications, additional language support, and enhanced reporting features. With further testing and deployment improvements, the system can become a more reliable and scalable solution for institutional communication.

.

8.3 ROLE-BASED FUNCTIONAL ANALYSIS

For administrators the primary result is a controlled notice publishing and management interface. For students the principal result is a searchable set of permitted active notices with bookmarking and notifications. The availability of these workflows can be demonstrated locally; adoption and institutional effectiveness have not been measured.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

8.4 NOTIFICATION RESPONSIVENESS DISCUSSION

Polling means there may be a short interval between publication and an alert appearing in an open browser. The project deliberately presents this as near-real-time rather than continuous push delivery. When email credentials are not configured, local job creation can be verified without claiming external inbox receipt.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

8.5 MULTILINGUAL ACCESS DISCUSSION

English–Malayalam translation and the source-based assistant support accessibility for different readers. Because generated translations can lose nuance, the application keeps the original source. A robust evaluation would use representative notices reviewed by bilingual readers and would report actual error types.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

8.6 SECURITY AND PRIVACY DISCUSSION

Server-side access checks, password hashing, CSRF and restricted attachments contribute to a safer demonstration than an unauthenticated shared notice directory. The implementation does not imply that every infrastructure control such as disk encryption or an external security audit is complete.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 9
CONCLUSION

9.1 SUMMARY

CollegeNotice is a web-based application developed to simplify and improve the management of institutional announcements. It provides a centralized platform where administrators can publish, update, and manage notices, while students and faculty members can access relevant information efficiently. The system incorporates department-based filtering, role-based access control, search functionality, notifications, and language assistance to improve communication and accessibility. Its structured approach reduces dependence on traditional notice boards and scattered communication channels. The project demonstrates how web technologies can support efficient information distribution, improve user convenience, and provide a foundation for modernizing institutional communication.

9.2 LIMITATIONS

The current implementation operates primarily in a local development environment and uses demonstration-based email notifications. Deploying the application in an actual institution requires additional configuration, including production hosting, database management, security measures, and email service integration. Public user registration also requires stronger verification mechanisms to ensure that only authorized institutional members can access the system. Translation functionality depends on the installation and configuration of the required language models, which may increase storage and processing requirements. The polling-based notification mechanism may introduce delays when delivering updates. These limitations should be addressed through further development, testing, and deployment improvements before institution-wide implementation.

9.3 FUTURE SCOPE

CollegeNotice can be enhanced with several features to improve its functionality, security, and usability. Institution-verified account provisioning can ensure that only authorized students, faculty members, and administrators access the platform. Real-time push notifications can provide faster updates about important announcements, examination schedules, and deadlines. A dedicated mobile application or mobile application wrapper can improve accessibility and provide a more convenient user experience. Advanced administrative approval workflows can introduce additional verification before important notices are published. Future improvements may also include analytics dashboards, notification delivery tracking, enhanced multilingual support, automated backups, and performance

monitoring. Comprehensive testing under different user loads can help evaluate scalability and reliability before large-scale deployment.

9.4 APPLICATIONS

CollegeNotice can be implemented in colleges, universities, and other educational institutions to manage announcements and distribute information efficiently. Academic departments can use the system to share lecture schedules, assignment deadlines, examination notifications, and departmental activities. Examination cells can publish examination timetables, registration deadlines, and result announcements. Student clubs and organizations can communicate information about events, competitions, workshops, and extracurricular activities. Libraries can use the platform to announce operating-hour changes, book availability updates, and other important services. Training centres can distribute course schedules, assessment dates, and administrative updates. The flexible architecture allows institutions to customize user roles, announcement categories, and access permissions according to their specific requirements.

9.5 IMPLEMENTATION BOUNDARIES AND NEXT STEPS

The completed mini project establishes a locally demonstrable notice management workflow with access control and optional full-stack services. Before campus rollout, representative users should review usability and language accuracy, and infrastructure operators should validate backup, hosting, email deliverability and worker recovery.

The engineering rationale is to keep policy decisions close to the database and request handlers. A client presentation can improve discoverability, but it is not sufficient evidence of a secure workflow. For each operation, the expected actor, input, output and failure behaviour should be identified before claiming the feature is complete.

An observable verification method is to establish a known sample notice and user role, perform the action through the relevant browser page or endpoint, and inspect both the response and resulting stored record. Negative-path verification is equally necessary: an invalid date, unrelated department, unauthorised attachment, or missing service must not silently produce a successful result.

This design also creates operational trade-offs. Local demonstration requires fewer external dependencies and is convenient for assessment, but the complete service profile requires a relational database, managed file storage, a task broker and worker availability. Reliability is therefore discussed separately from the functions that can be shown on a development machine.

The report treats the uploaded source files as the evidence of implementation and the review presentation as the description of intended behaviour. Where the presentation uses broader language than the source supports, the narrower tested behaviour is reported. This approach makes the document useful for both project evaluation and future maintenance.

From the student perspective, the important property is that a notice is relevant, current and traceable to an authorised institutional publisher. From the staff perspective, the same design needs a reproducible publishing process and an auditable record of changes. Both perspectives should be included when judging a requirement or planning a later improvement.

CHAPTER 10
APPENDICES

SAMPLE SOURCE – ROLE-CONSTRAINED NOTICE ACCESS

```
import uuid
from pathlib import Path
from django.contrib.auth.models import AbstractUser
from django.db import models
from django.utils import timezone

class Department(models.Model):
    name = models.CharField(max_length=100, unique=True)
    description = models.CharField(max_length=250, blank=True)
    class Meta:
        ordering = ['name']
    def __str__(self):
        return self.name

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    description = models.CharField(max_length=250, blank=True)
    class Meta:
        ordering = ['name']
    def __str__(self):
        return self.name

class User(AbstractUser):
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=16, choices=[('student', 'Student'), ('admin', 'Department
admin')], default='student')
    department = models.ForeignKey(Department, null=True, blank=True,
on_delete=models.PROTECT)
    email_alerts = models.BooleanField(default=True)
```

```
def attachment_path(instance, filename):
    return f'attachments/{uuid.uuid4().hex}{Path(filename).suffix.lower()}'

class NoticeQuerySet(models.QuerySet):
    def active(self):
        return self.filter(archived=False, publish_date__lte=timezone.localdate(),
expiry_date__gte=timezone.localdate())

class Notice(models.Model):
    title = models.CharField(max_length=180)
    body = models.TextField(max_length=20000)
    summary = models.TextField(blank=True)
    translation = models.TextField(blank=True)
    ai_status = models.CharField(max_length=30, default='local')
    priority = models.BooleanField(default=False)
    priority_reason = models.CharField(max_length=250, blank=True)
    force_priority = models.BooleanField(default=False)
    department = models.ForeignKey(Department, null=True, blank=True,
on_delete=models.PROTECT)
```

CHAPTER 11
REFERENCES

11.1 IMPLEMENTATION REFERENCES

1. Celery Project. (2026). Celery documentation. https://docs.celeryq.dev/

2. CollegeNotice development team. (2026). CollegeNotice: Source code, architecture notes and local validation records [Unpublished project source archive].

3. Costa-jussà, M. R., et al. (2022). No language left behind: Scaling human-centered machine translation. arXiv. https://arxiv.org/abs/2207.04672

4. CT2 Contributors. (2026). CTranslate2 documentation. https://opennmt.net/CTranslate2/

5. Django REST Framework. (2026). API guide. https://www.django-rest-framework.org/api-guide/

6. Django Software Foundation. (2026). Django documentation, version 5.2. https://docs.djangoproject.com/en/5.2/

7. Hugging Face. (2026). NLLB model documentation. https://huggingface.co/docs/transformers/model_doc/nllb

8. Internet Engineering Task Force. (2015). JSON Web Token (JWT) (RFC 7519). https://www.rfc-editor.org/rfc/rfc7519

9. Internet Engineering Task Force. (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). https://www.rfc-editor.org/rfc/rfc8259

10. Internet Engineering Task Force. (2022). HTTP semantics (RFC 9110). https://www.rfc-editor.org/rfc/rfc9110

11. Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

12. Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. Proceedings of EMNLP: System Demonstrations, 66–71. https://aclanthology.org/D18-2012/

13. Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into text. Proceedings of EMNLP 2004, 404–411. https://aclanthology.org/W04-3252/

14. Mozilla Developer Network. (2026). Progressive web apps. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

15. National Institute of Standards and Technology. (2025). Digital identity guidelines: Authentication and authenticator management (SP 800-63B). https://pages.nist.gov/800-63-4/sp800-63b.html

16. Ninawe, M., Sarangpure, P., Sahare, P., Shende, D., & Yadhav, M. (2025). Web-Controlled Centralized Digital Notice Board for Smart Institutions. International Journal of Electrical, Electronics and Computer Systems, 14(1), 91–97. https://doi.org/10.65521/ijeecs.v14i1.351

17. OWASP Foundation. (2026). Authorization cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

18. OWASP Foundation. (2026). Cross-site request forgery prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

19. OWASP Foundation. (2026). File upload cheat sheet.
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
20. PostgreSQL Global Development Group. (2026). PostgreSQL documentation.
https://www.postgresql.org/docs/
21. Python Software Foundation. (2026). Python 3 documentation. https://docs.python.org/3/
22. Redis Ltd. (2026). Redis documentation. https://redis.io/docs/latest/
23. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. IEEE Computer, 29(2), 38–47. https://doi.org/10.1109/2.485845
24. SQLite Consortium. (2026). SQLite documentation. https://www.sqlite.org/docs.html
25. World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2.
https://www.w3.org/TR/WCAG22/

Nithin Mathew

COLLEGE_NOTICE_BORAD_REPORT

mini project report

Document Details

Submission ID

trn:oid:::10159:154691316

Submission Date

Oct 14, 2026, 9:16 PM GMT+5:30

Download Date

Oct 14, 2026, 9:21 PM GMT+5:30

File Name

COLLEGE_NOTICE_BORAD_REPORT.pdf

File Size

1.8 MB

77 Pages

17,944 Words

111,186 Characters

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

8 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

2 Missing Quotations 3%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
3% Publications
6% Submitted works (Student Papers)